DESIGN QUESTIONS

Design a Real-Time Collaborative Editor

Operational Transform, CRDTs & Multi-User Conflict Resolution

~ 35 minute read

© Study Guide — For personal use only

What's Inside

01 · Problem Overview & Scale
02 · Clarifying Questions
03 · Capacity Estimation
04 · High-Level System Architecture
05 · Core Design Decisions: OT vs CRDT
06 · Data Models & Storage
07 · Concurrent Edit Processing
08 · OT/CRDT Deep Dive
09 · Presence & Cursor Tracking
10 · Document Load, Save & Offline Mode
11 · Failure Modes & Recovery
12 · Interview Playbook

 © Study Guide — For personal use only

01 Problem Overview & Scale

Design a real-time collaborative document editor similar to Google Docs.

- **1 billion documents** stored across all users
- **100 million daily active users** opening, editing, and sharing documents
- **50 concurrent users** editing the same document simultaneously (typical; can spike higher)
- **Sub-second latency** for seeing others' edits (cursor positions, text changes)
- **Offline support:** Users can edit without internet; sync when reconnecting
- **Conflict resolution:** Concurrent edits from multiple users must merge correctly

Key Challenges

- **Consistency:** Ensure final document state is identical across all clients (eventual consistency)
- **Low latency:** Optimistic updates on the client for instant UI response
- **Conflict resolution:** When two users insert text at the same position, one must win deterministically
- **Scalability:** Support millions of documents, billions of edits, high-throughput revision logs
- **Undo/Redo:** Per-user undo must not affect others' edits
- **Offline & sync:** Buffer ops locally (IndexedDB), merge on reconnect

02 Clarifying Questions

Scope	Question	Assumption
Scope	Real-time consistency requirement?	Eventual consistency; broadcast within 500ms–1s
Strategy	Conflict resolution approach?	Operational Transform (simpler) or CRDT (better at scale)
Offline	Must support offline editing?	Yes; sync on reconnect with conflict detection
Size	Document size limits?	Up to 50MB per document (typical: <1MB for collaborative editing)
History	Revision history retention?	Last 30 days of full history; snapshots every 100 edits
Users	Max concurrent users per doc?	Design for 50; graceful degradation up to 100+
Presence	Show cursor positions & user list?	Yes; real-time presence, color-coded cursors, typing indicators

© Study Guide — For personal use only

Scope	Question	Assumption
Permissions	Access control needed?	Yes; owner, editor, viewer roles; share permissions

03 Capacity Estimation

Concurrent Sessions & Operations

- **100M DAU × 10% concurrent:** 10M users online at peak
- **Avg 3 docs per user session:** ~30M active document sessions at peak
- **50 users per doc (median; 90th percentile 100+):** Key bottleneck = document session
- **Per-user edit rate:** 1–5 operations/second (typing, delete, format changes)
- **Peak edit throughput:** 30M docs × 2 ops/sec avg = 60M ops/sec globally

Bandwidth & Latency

- **Op size:** ~200 bytes per operation (JSON: type, pos, text, user_id, timestamp)
- **Broadcast fanout:** 50 concurrent users per doc → 50 × 200 bytes = 10KB per op per doc
- **Peak inbound:** 60M ops/sec × 200 bytes = 12 TB/sec (aggregated across regions)
- **Presence updates:** 1 cursor/user/100ms = 500 cursors/doc per second at 50 users
- **Target latency:** Optimistic update immediate (0ms); server ack <100ms; peer ops <500ms

Storage & Revision History

- **Document snapshots:** 1B docs × 50KB avg = 50PB (compressed: ~20PB with gzip)
- **Revision log:** 1 doc may have 10K–100K ops over lifetime; 1B docs × 50K ops avg × 200 bytes = 10EB (archived/pruned)
- **Hot storage (last 30 days):** ~100M active docs × 100 ops/day × 200 bytes = 2PB
- **Index:** {doc_id, user_id, timestamp} for revision lookup; sharded by doc_id

04 High-Level System Architecture

The system is divided into three main layers:

© Study Guide — For personal use only

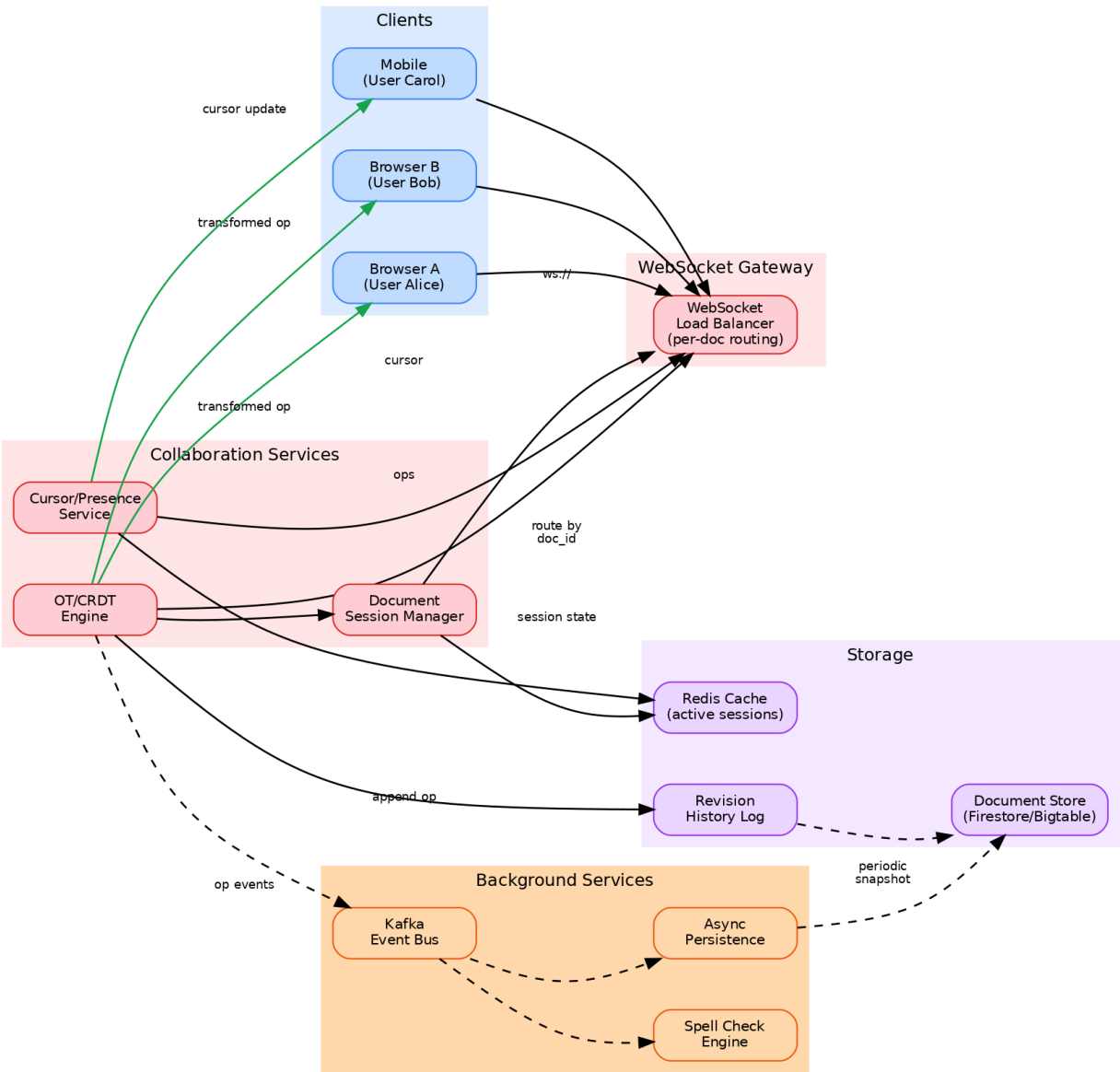

Figure 4.1: High-Level System Architecture with Clients, WebSocket Gateway, Collaboration Services, Storage, and Background Services.

Client Layer

- **Web/Mobile clients:** Browser or native app; maintains local doc state in memory
- **WebSocket connection:** Persistent bidirectional channel to collaboration server
- **Local cache (IndexedDB):** Store snapshot + recent ops for offline mode
- **Optimistic updates:** Apply edits immediately to UI while sending to server

WebSocket Gateway & Load Balancing

- **Per-document routing:** All users editing doc_id=X route to same server instance (sticky session)
- **Horizontal scaling:** Hash(doc_id) → server instance; scale by sharding docs across backends
- **Connection multiplexing:** One WebSocket per client; multiple docs per session via msg routing

© Study Guide — For personal use only

- **Heartbeat & reconnection:** Detect stale connections; fast reconnect with op backfill

Collaboration Services

- **OT/CRDT Engine:** Transform incoming ops against concurrent ops; compute final state
- **Document Session Manager:** Maintain active doc state, session metadata, user list per doc
- **Cursor/Presence Service:** Track real-time cursor positions, typing indicators, user colors

Storage Layer

- **Document Store (Firestore/Bigtable):** Persistent snapshots; schema: {doc_id, snapshot, version, timestamp}
- **Revision Log:** Append-only log of operations; index by {doc_id, version} for quick access
- **Redis Cache:** Hot session state (active user list, last known version, in-flight ops)

Background Services (Async)

- **Async Persistence:** Periodically save snapshots (every 100 ops or 30 seconds)
- **Spell Check Engine:** Real-time spell checking; flag errors without blocking edits
- **Notification Service (Kafka):** Broadcast doc changes, @mentions, permission updates

05 Core Design Decisions: OT vs CRDT

The choice between Operational Transform (OT) and Conflict-free Replicated Data Types (CRDT) is critical.

Aspect	Operational Transform (OT)	CRDT
Consistency	Strong eventual consistency; requires central server	Weak eventual consistency; decentralized possible
Transformation	Must compute transform(op_a, op_b); non-commutative	Ops commute; no transform needed
Offline support	Harder; must detect & resolve conflicts on reconnect	Natural; automatic merge via unique IDs
Complexity	Complex xform logic; hard to implement correctly	Simpler to reason about; deterministic
Performance	Fast for small N users; slower with many concurrent edits	Consistent performance; scales well
Undo/Redo	Complex; must track op dependencies	Simpler; each op has unique ID
Production use	Google Docs, Wave (centralized)	Figma, Apple Notes (local-first)

© Study Guide — For personal use only

Recommended Approach: Hybrid (Central OT)

- **Primary:** Central server-side OT engine for authoritative conflict resolution
- **Backup:** Client-side CRDT-like merge for offline sync and eventual consistency fallback
- **Rationale:** OT + central server = predictable, testable, audit-friendly (Google Docs approach)
- **Scaling:** Shard by doc_id to keep OT engine simple per shard; docs don't move between servers

✓ DESIGN DECISION

We will use **Operational Transform** with a **central OT engine** and **sticky sessions per doc**. All edits route to one server; that server transforms ops and broadcasts results. This trades some scalability (can't horizontally scale per doc) for simplicity and correctness. At 50 users per doc, a single modern server (16 cores, 64GB RAM) can handle ~1000 concurrent docs.

06 Data Models & Storage

Document Entity

```
message Document {
string doc_id = 1; // unique ID (UUID)
string title = 2;
string content = 3; // current full text (UTF-8)
int64 version = 4; // Lamport version counter
Timestamp created_at = 5;
Timestamp updated_at = 6;
string owner_user_id = 7;
repeated Permission permissions = 8; // role: owner/editor/viewer
}
```

Core document metadata stored in Firestore/Bigtable

Operation Log Entry

```
message Operation {
string doc_id = 1;
int64 version = 2; // monotonic version for this doc
string user_id = 3;
Timestamp timestamp = 4;
string type = 5; // 'insert' | 'delete' | 'format'
int32 position = 6; // char position in document
```


© Study Guide — For personal use only

```
string content = 7; // inserted text (for insert)
int32 length = 8; // chars deleted (for delete)
map<string, string> attrs = 9; // format attrs {bold, italic, ...}
}
```

Immutable operation logged in append-only revision log

Session & Cursor State (Redis)

```
// Redis key: session:{doc_id}
{
version: 12500, // latest applied op version
users: {
alice: {user_id, cursor_pos, color, typing: true},
bob: {user_id, cursor_pos, color, typing: false}
},
pending_ops: [
{version: 12501, op_obj}
{version: 12502, op_obj}
]
}
```

Volatile session state for active document editing

Snapshot Storage Strategy

- **Periodic snapshots:** Every 100 operations or 30 seconds; store {doc_id, snapshot_version, content, timestamp}
- **Snapshot + log:** On load, fetch latest snapshot, then replay ops from that version forward
- **Log cleanup:** Archive ops older than 30 days; keep only last 3 snapshots per doc to save space
- **Compression:** Store snapshots as gzipped JSON; 50KB avg → 5KB compressed

07 Concurrent Edit Processing

The heart of the system: how concurrent edits from multiple users are transformed and merged.

© Study Guide — For personal use only

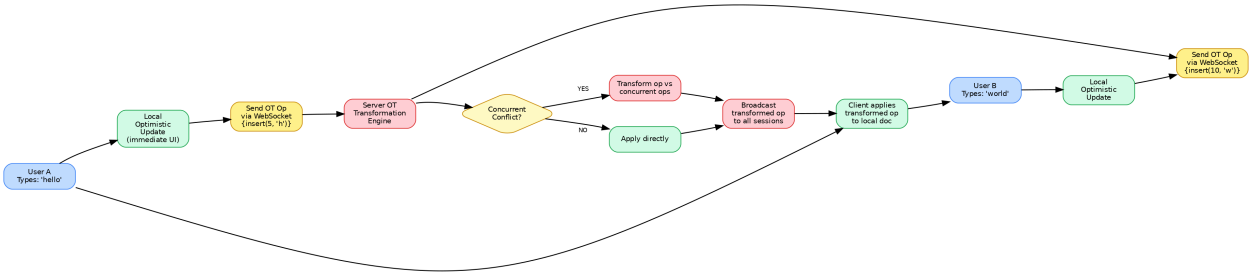

Figure 7.1: Concurrent Edit Processing: Two users typing simultaneously, ops sent to server, OT engine transforms, then broadcasts transformed ops to all clients.

The OT Transformation Algorithm (Simplified)

- **Scenario:** User A inserts 'hello' at position 5; User B inserts 'world' at position 10. Both arrive at server.
- **Server state before either op:** version=100, content='previous text'
- **Op A arrives:** insert(pos=5, 'hello'), applied → version=101, broadcast to B
- **Op B arrives:** insert(pos=10, 'world'), but conflicts with A's position shift
- **Transform step:** Adjust B's position against A: pos = 10 + len('hello') = 15 (because A's insertion shifts everything after)
- **Apply transformed B:** insert(pos=15, 'world'), applied → version=102
- **Broadcast:** Both A and B receive transformed ops; convergence guaranteed

Why This Works (Convergence Proof)

- **Commutativity:** If transform function is commutative, final state is independent of op order
- **Idempotency:** A client that sent op_i won't re-apply it when broadcast back (dedup by op_id)
- **Causality:** Each op includes a version number; ops must be applied in version order

> **N OT TRANSFORMATION**
>
> The key invariant: **transform(op_a, op_b) ≡ transform(op_b, op_a)** (up to reordering). This ensures all clients converge to the same final state even if they apply ops in different orders. The server is authoritative; it decides version order.

08 OT/CRDT Deep Dive

Operational Transform (OT) Essentials

- **Core idea:** Given two concurrent ops, compute a third op that applies both edits correctly
- **Transform function:** op' = T(op_a, op_b) where op' is op_a adjusted for op_b's effects
- **For insert operations:** If op_b inserts before op_a's position, shift op_a's position forward
- **For delete operations:** Delete positions shift based on prior insertions; overlapping deletes handled carefully

© Study Guide — For personal use only

- **String vs tree:** Most collaborative editors use string (linear sequence); complex docs use tree (nodes, hierarchy)

CRDT (Conflict-free Replicated Data Type) Alternative

- **Core idea:** Assign unique IDs to each character; no transform needed because order is via ID comparison
- **Example — Yjs CRDT:** Each char gets {unique_client_id, sequence_num}; merge sorts by ID
- **Advantage:** Works offline; automatic eventual consistency; simpler merge logic
- **Disadvantage:** Higher memory overhead (IDs per char); harder to implement undo/redo per-user
- **Best for:** Local-first apps (Figma, Apple Notes); eventual consistency acceptable

Practical Trade-Offs

- **OT:** Smaller payloads, faster (no metadata per char), audit-friendly (linear history); needs central server
- **CRDT:** Scales offline editing, supports true P2P, deterministic merge; larger payloads, complex logic
- **Hybrid approach:** Use OT on the server for consistency; clients cache CRDT-style IDs for offline merge

09 Presence & Cursor Tracking

Real-Time Cursor Positions

- **Update frequency:** Send cursor position every 100ms (or on significant movement) to avoid flooding
- **Payload:** {user_id, cursor_pos, selection_start, selection_end, color} ~50 bytes
- **Broadcast:** Server multicasts to all other users in the session (fanout to N-1 clients)
- **Latency:** Target <200ms for cursor visibility

Presence Heartbeat & Timeout

- **Heartbeat:** Client sends keepalive every 30 seconds; includes user status (online, away, typing)
- **Server timeout:** Remove user from active list if no heartbeat for 2 minutes (network stale)
- **Graceful exit:** User closes editor → send 'user left' event → remove from all clients' presence list

User Colors & Avatars

- **Color assignment:** On session join, server picks next color from palette (8–10 colors, rotate through users)
- **Avatar display:** Show user initials + color in sidebar and next to cursor
- **Consistency:** Same user always same color across all docs and sessions

© Study Guide — For personal use only

10 Document Load, Save & Offline Mode

The read/persistence path and offline sync strategy.

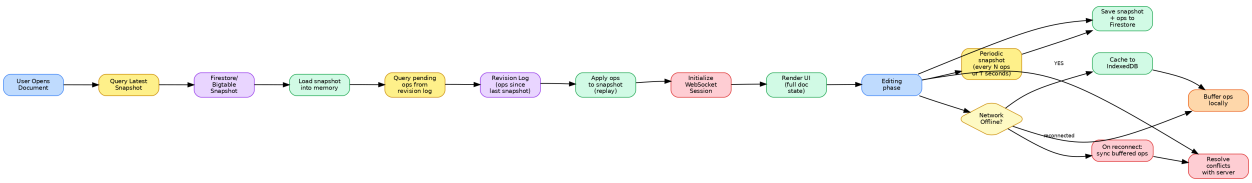

Figure 10.1: Document Load, Periodic Save, and Offline Sync Flow.

Loading a Document

- **Step 1:** User opens doc → fetch latest snapshot from Firestore (indexed by doc_id)
- **Step 2:** Query revision log: get all ops since snapshot version
- **Step 3:** Replay ops on snapshot in version order → current document state
- **Step 4:** Initialize WebSocket session; subscribe to real-time ops for this doc
- **Step 5:** Render UI with full document + user list

Periodic Snapshot & Save

- **Trigger:** Every 100 operations OR 30 seconds, whichever comes first
- **Process:** Server saves {doc_id, snapshot_content, version, timestamp} to Firestore
- **Log cleanup:** Ops older than snapshot version can be archived (keep 30 days for audit)
- **Latency:** Async; doesn't block real-time editing

Offline Mode & Reconnection

- **On disconnect:** Client detects WebSocket close; stop sending ops to server
- **Local buffering:** Cache all edits to IndexedDB (browser's local database)
- **Optimistic UI:** Continue showing edits in UI immediately (local only)
- **Reconnection detect:** Periodically try to reconnect (exponential backoff)
- **Sync on reconnect:** Send buffered ops to server; server transforms against concurrent edits during offline period
- **Conflict resolution:** If conflict detected, show user merge UI or accept server version

> ■ **OFFLINE DESIGN**
>
> Offline is supported via local IndexedDB cache + buffered ops. When reconnecting, send buffered ops with version info. Server compares against its state; if diverged, apply transformation or ask client to resolve. For most use cases (single user editing offline), automatic merge succeeds.

© Study Guide — For personal use only

11 Failure Modes & Recovery

Failure Mode	Impact	Recovery Strategy
WebSocket disconnect	Ops don't reach server; client unaware	Client detects close event; buffer ops; exponential backoff reconnect
Server crashes	All active sessions lost; pending ops may be lost	Failover to hot standby (Redis backup); clients reconnect; re-sync from snapshot + log
Op loss during persist	Ops applied to docs but not persisted	Periodic snapshots are synchronous/durable; ops logged to journal before ACK
Split-brain (network partition)	Clients diverge; can't reach server	Clients buffer ops locally; on reconnect, server version wins; client merges or discards local ops
Concurrent op collision	Same position edited by 2+ users simultaneously	OT transform resolves; server version order is authoritative
Slow client (lagging)	Client receives ops out of order or with gaps	Client queues ops by version; applies once version is contiguous

12 Interview Playbook

Structure for a 45-minute interview on real-time collaborative editing.

Time	Topic	What to Cover	Depth
0–5 min	Problem statement	1B docs, 100M DAU, 50 users/doc, sub-second latency	Clarifying questions
5–10 min	Requirements	Functional (edit, share, history) & non-functional (99.9% availability, <500ms op latency)	Trade-off discussion
10–15 min	Capacity estimation	60M ops/sec, 50PB snapshots, 2PB hot storage, per-doc session sharding	Back-of-envelope math
15–25 min	High-level architecture	WebSocket gateway, OT engine, doc store (Firestore/Bigtable), Redis session cache	Draw all 3 diagrams
25–30 min	OT vs CRDT	Compare approaches; justify OT for this use case (central + sticky sessions)	Design decision
30–35 min	Deep dive: one topic	OT algorithm & transform, or presence/cursors, or offline sync	Interviewer picks

© Study Guide — For personal use only

Time	Topic	What to Cover	Depth
35–40 min	Failure scenarios	Handle WebSocket drops, server crashes, op loss, merge conflicts	Resilience discussion
40–45 min	Wrap-up & trade-offs	Scalability limits (per-doc shard), latency (network RTT), cost (storage)	Summary & questions

Key Points to Emphasize

- **Scalability:** Shard by doc_id (sticky sessions); each shard can handle ~1000 docs
- **Consistency:** Strong eventual consistency via OT; server authoritative
- **Latency:** Optimistic updates (0ms), server ack (<100ms), broadcast (<500ms)
- **Reliability:** Async persistence; hot failover; offline sync on reconnect
- **Trade-offs:** OT simpler than CRDT but requires central server; not ideal for P2P

N KEY TAKEAWAYS

Real-time collaborative editing hinges on **Operational Transform** for conflict resolution and **central server authority** for consistency. Use **sticky sessions per doc** to avoid distributed consensus. Store **snapshots + op logs** for durability and recovery. Support **offline editing via local cache** and merge on reconnect. Design for **50 concurrent users per doc** with sub-500ms op propagation latency.

© Study Guide — For personal use only

Further Reading & References

- **Google Docs architecture:** https://www.usenix.org/conference/osdi10/real-time-collaborative-editing-wave
- **Operational Transform paper (Ellis & Gibbs, 1989):** Foundational OT work
- **CRDT intro (Yjs library):** https://github.com/yjs/yjs; showcase of CRDT-based editing
- **Figma's multiplayer tech:** https://www.figma.com/blog/how-figmas-multiplayer-technology-works/

© Study Guide — For personal use only